names
bindings
scopes

ADEVA | AQUINO | ESPILOY | LAMBAYAO | LUMAPAS | NUÑAL | SABINO

outline

1 NAMES

2 VARIABLES

3 THE CONCEPT OF BINDING

4 STORAGE BINDINGS AND LIFETIME

5 SCOPE

6 SCOPE AND LIFETIME

7 REFERENCING ENVIRONMENTS

8 NAMED CONSTANTS

names

- Also known as an **identifier** (such as x, total, count), is a symbolic label in the program that you use to refer to something else, such as a variable, function, class, or module.

- It is one of the **most fundamental attributes of variables;** however,

- They are also associated with some program constructs, such as **labels, subprograms, and formal parameters.**

names

Example:

int **x** = 5;

double **price** = 10.99;

char **grade** = ‘A’;

In each line, the identifier on the left (such as x, price, or grade) is the name of a variable, the keyword on the left (int, double, char) specifies its type, and the value on the right is the initial value stored in that variable.

x, price, and grade are not the values themselves but rather ways to talk about the variable later.

primary design issues for Names

- What is the **maximum length** of a name?
- Can **connector characters** be used in names?
- Are names **case sensitive**?
- Are the special words **reserved words or keywords**?

maximum length

- The earliest programming languages used single characters
- FORTRAN I and FORTRAN 77: up to 6 characters
- FORTRAN 90 AND C: up to 31 characters
- C++: no limit, but implementors sometimes put limit

maximum length

Earliest PLs:

A = 5

FORTRAN I and FORTRAN 77:

TOTAL1 = 100

SUM123 = 50

TOTALSUM = 100 *! invalid since more than 6*

FORTRAN 90 and C:

int totalStudentCount = 50;

float averageScoreValue = 89.5;

Java and Ada:

int totalNumberOfStudentsInTheClassroom = 50;

connector characters

- while most contemporary languages allow connector characters in names
- other languages such as Pascal, Modula-2, FORTRAN 77, don't allow connector characters in names

```
totalscore          // correct
total_score         //  also correct (more readable)

total-score = 90    # error because dash is treated as minus operator

TOTAL_SCORE = 100   ! not allowed in older FORTRAN
TOTALSCORE = 100
```

case sensitive

- C, C++, Java: names are case sensitive
- Versions of FORTRAN prior to 90, only uppercase letters are allowed in names
- Some languages are **not** case sensitive

```
apple != APPLE != Apple          // case sensitive

orange = ORANGE = Orange         // not case sensitive
```

special words

- reserved words or keywords
- **Special words** in PLs are used to make programs more readable by naming actions to be performed.
- A **keyword** is a word of a PL that is special only in certain contexts.
- A **reserved word** is a special word of a PL that cannot be used as a name.

special words

Example:

REAL is a special word.

The meaning of **REAL** depends on its context

REAL APPLE	*REAL is a keyword*
REAL = 1.5	*REAL is a variable name*

therefore, in this context, real can mean a datatype or a variable name

special words

- as a language design choice, **reserved words are better than keywords** because the ability to redefine keywords can lead to readability problems.

Example: FORTRAN INTEGER and REAL

INTEGER REAL
REAL INTEGER

the appearance of INTEGER and REAL elsewhere in the program could be misleading to program readers

special words

Example:

int float = 5 *# invalid*

The compiler gets confused because you're trying to use a keyword as a variable name.

Predefined Names: Already given a meaning by the language BUT can still be redefined (overwritten) by the user.

print = 10 *# normally a built-in function, but here you redefined it as a variable.*

variables

- It is the programming language's abstraction of a computer memory cell. A variable is not only one thing; it is characterized by its multiple attributes, which are:

1. Name
2. Address
3. Type
4. Value
5. Scope
6. Lifetime

name

- As mentioned before, a string of characters or identifier that the programmer refers to the variable with in the code.

Example:

int **score** = 10;

The name is **score**. When you later modify or do operations with value, the compiler knows you are referring to that variable.

address

- The memory location where the variable is associated with.

Example:

```
int score = 10;
printf("%p", &score);   // prints something like 0x7ffeeb2c3a4c
```

The address is the shown to us by the output of the “printf("%p", &score)”, so when the score variable was created, the CPU writes it in that address.

type

- Defines the range of values the variable has and the operations that can be performed on it.

Example:

```
int age = 25;    // type int → whole numbers, supports +, -, *, /, %
float price = 19.99; // type float → decimal numbers, supports +, -, *, /
char grade = 'A';  // type char → single character, supports comparison
```

The type is the “int, float, and char” in the code snippet; it determines which values it accepts and operations it can do.

value

- The actual data stored in the memory cell.

Example:

```
int counter = 0;  // value is 0
counter = counter + 3; // now value is 3
```

The value is the actual content stored in the memory cell, which in this case, the value of the counter started at 0 and then turned to 3 since 3 was added to the variable.

scope

- The region where the program source code of the variable is accessible by its name.

Example:

```
int x = 10;      // global scope: visible everywhere in this file

void func() {
    int y = 20;    // function scope: visible only inside func()
    if (y > 0) {
        int z = 30;  // block scope: visible only inside this if block
        x = x + z;  // ok: x is global, z is visible here
    }
    // z is NOT visible here – compilation error if you use z
}
```

lifetime

- The period when the variable is going to a specific memory location.

Example:

```
int global = 100;    // static lifetime: exists entire program run

void myFunction() {
    int local = 5;   // stack lifetime: created when function is called,
                     // destroyed when function returns
    static int keeper = 0; // static lifetime: retains value across calls
    keeper++;
}
```

the concept of binding

- A **binding** is an association between an attribute and an entity or between an operation and a symbol.
- For example, binding a variable to its type, or a name to a specific memory location

- The **Binding Time** is simply the moment when that association (**binding**) takes place

binding time

- Language design time

  binding **operator symbol** (+) to an **operation** (addition)

- Language implementation time / Compiler design time

  bind **type** to **range of possible values**

- Compile time

  binding a variable to a **type**

- Link time

  binding a **function call** to a library routine

- Load time

  binding a call to a **library subprogram** to the subprogram code

- Runtime

  binding **variables**, declared inside a function in C, to a type

binding time

Example

```
int count ;
count = count + 5
```

binding time

```
int count ;
count = count + 5
```

The set of types for count

bound at language design time

The type of count

bound at compile time

The set of possible values for count

bound at compile design time

The value of count

bound at runtime

static binding

- Occurs before the program runs (at **compile time**) and remains unchanged throughout execution.
- It is generally faster because the computer doesn't have to "figure things out" while the program is running.

dynamic binding

- Occurs during execution (at **runtime**). It can change as the program flows, offering more flexibility but typically requiring more overhead (processing power).

static type binding

- In static type binding, the type of a variable is determined before the program runs (at **compile time**) and stays the same for the entire life of the program.

Explicit declaration

- means of associating variables with types through declaration statements.
- The programmer manually writes the type in the code.

Java / C++

```
int count = 10 ;
```

The compiler sees **int** and immediately binds **count** to the integer type.

static type binding

- In static type binding, the type of a variable is determined before the program runs (at **compile time**) and stays the same for the entire life of the program.

Implicit declaration

- means of associating variables with types through default conventions instead of declaration statements.
- The first appearance of a variable name in a program constitutes its implicit declaration.
- The compiler "guesses" or infers the type based on the value assigned to it, but once it decides, the type is **locked**.

`

static type binding

- In static type binding, the type of a variable is determined **before the program runs** (at **compile time**) and stays the same for the entire life of the program.

Implicit declaration

Javascript

```
let message = “Hello” ;
```

Even though you didn't type String, the compiler binds message to a String at compile time. You cannot change it to an integer later.

static type binding

Implicit declaration has **RISKS**

The Risk:

Typographical errors (e.g., totall = total + 1) create new, invisible variables that lead to silent logic bugs.

The Solution (Sigils):

Perl uses **$** for **scalars** and **@** for **arrays** so the compiler can **verify the intent** of the variable even if it wasn't explicitly declared.

The Solution (Strict Rules):

Using **implicit none** (Fortran) or **'use strict'** (JS) forces the binding to move from "Implicit/Dynamic" to "Explicit/Static," which drastically improves code reliability.

dynamic type binding

- In dynamic type binding, the type of a variable is determined **during execution** (at **run time**). The variable is essentially a "container" that can hold any type of data at any time.
- The type is bound to the **value**, not the name. The name just points to whatever value (and its associated type) is currently assigned.
- This provides programming flexibility – capable of dealing with any type of data.
- This can be costly as type checking is done at runtime.
- Type error detection by the compiler is difficult.

dynamic type binding

- In dynamic type binding, the type of a variable is determined **during execution** (at **run time**). The variable is essentially a "container" that can hold any type of data at any time.
- The type is bound to the **value**, not the name. The name just points to whatever value (and its associated type) is currently assigned.
- This provides programming flexibility – capable of dealing with any type of data.
- This can be costly as type checking is done at runtime.
- Type error detection by the compiler is difficult.

storage bindings and lifetime

Fundamentals of **Storage Binding**

- **Allocation**: The process of binding a variable to a memory cell from a pool of available memory.
- **Deallocation**: The process of unbinding a variable and returning that memory cell to the pool.
- **Lifetime**: The duration of time a variable remains bound to a specific memory location.
  - Starts: When the variable is bound to a cell.
  - Ends: When the variable is unbound.

```
{
  int x; // Allocation happens here
  x = 5;
} // Deallocation happens here
```

storage bindings and lifetime

Types of Lifetime

- Static Variables
- Stack-Dynamic Variaables
- Explicit Heap-Dynamic Variables
- Implicit Heap-Dynamic Variabes

storage bindings and lifetime

Static Variables

Static variables are the "permanent residents" of memory. They are bound to a specific memory cell before the program begins and stay in that exact spot until the program closes.

- History Sensitivity: This is their most unique trait. Because they are never deallocated during execution, they "remember" their state. If a function increments a static variable and finishes, the next time that function is called, the variable starts at the incremented value, not zero.
- Efficiency: They are the fastest variables to access because the compiler uses "direct addressing"—it knows the exact physical address from the start.
- The Trade-off: You lose flexibility. Specifically, you cannot use recursion with static variables because every recursive call would try to overwrite the same single memory cell instead of creating a new one.

storage bindings and lifetime

Stack-Dynamic Variables

- Definition: Storage binding occurs when their declaration is "elaborated" (reached) during execution.
- Pros:
  - **Allows** for recursion.
  - Saves memory by **sharing space** between subprograms.
- Cons:
  - **Not history sensitive** (values are lost when the function ends).
  - Slower access due to indirect addressing.

```
void greet(int n) {
    int temp = n; // Stack-dynamic
    if (n > 0) greet(n - 1); // Recursion
}
```

storage bindings and lifetime

Explicit Heap-Dynamic Variables

- Definition: Nameless memory cells allocated/deallocated by explicit instructions from the programmer.
- Mechanism: Managed via pointers or reference variables.
- Management:
  - C++: Uses new (allocation) and delete (deallocation).
  - Java: All objects are heap-dynamic, but deallocation is handled by Garbage Collection.

```
int *intnode;        // Pointer
intnode = new int;   // Allocation
delete intnode;      // Deallocation
```

storage bindings and lifetime

Implicit Heap-Dynamic Variables

- Definition: Bound to heap storage only when assigned a value; attributes are re-bound every time a new value is assigned.
- Pros:
  - Highest flexibility; allows for highly generic code.
  - Size and type are dynamic.
- Cons:
  - High runtime overhead.
  - Loss of error detection by the compiler.

```
x = 10;      // Bound as int
x = [1,2,3]; // Re-bound as array
```

scope

- The scope of a variable is the range of statements in which a varaible is **visible**.
  - A variable is **visible** in a statement if it can be referenced in that statement

Scope rules of a language

- It determines how a particular occurance of a name is associated with a variable
- Determine how references to variables declared outside the currently executing subprograms or block are associated with their declarations and thus their attributes.

Local Scope

A variable is local in a program unit (function or subprogram) or block if it is declared there

Nonlocal Scope

The nonlocal variable of a program unit or block are those that are visible within the program unit or block but are not declared there

Global Scope

Special case of nonlocal scope. These are names that are declared at the outermost region.

static scoping

It is the method of binding names to nonlocal variables

Scope can be statically (prior to execution) determined.

- To **connect a name reference** to a varible,

  the compiler must find the declaration

- When the declaration is found by the compiler,

  all the other attributes can be determined.

In ***static scoped languages*** **with nested subprograms**, the search process is as follows:

- For a given variable name inside a subprograms, locally search the declaration of that names inside the subprogram
- if no declaration is found locally, search in increasingly larger enclosing scopes until a declaration is found.

Enclosing static scopes (to a specific scope) are called its **static ancestors**
the nearest static ancestor is called a **static parent**

- If a local declaration of a variable is **found inside the subproblem**, the same variable declared at the static parent or ancestor of the subprogram is hidden from the code of the subprogram

Search for declaration of x

```
procedure big;
  var x : integer;
  procedure sub1;
    begin { sub1 };
    . . . x . . .
    end; { sub1 }
```

```
procedure sub2;
  var x : integer;
  begin { sub2 };
  . . .
  end; { sub2 }
begin { big }
. . .
end; { big }
```

- A declaration of a variable effectively hides any declaration of a variable with the same name in a larger enclosing scope

```
program main;
  var x : integer;
  procedure sub1;
    var x : integer;
    begin { sub1 };
    . . . x . . .
    end; { sub1 }
  begin { main }
  . . .
  end. { main }
```

dynamic scoping

It is based on the **calling sequence** of subprograms, not on their spacial relationship to each other

Scope is determined at runtime.

Search Process:

for a variable inside subprogram A

- search locally for the declaration
- if declaration not found, search in the subprogram that called the subprogram A

Search for declaration of x

Call sequence

- big > sub2 > sub1
- big > sub 1

```
procedure big;
  var x : integer;
  procedure sub1;
    begin { sub1 };
    . . . x . . .
    end; { sub1 }
```

```
procedure sub2;
  var x : integer;
  begin { sub2 };
  . . .
  end; { sub2 }
begin { big }
. . .
end; { big }
```

scope and lifetime are not the same!

Scope

- the range of statements in which a variable can be referenced
- a spatial concept

Lifetime

- the length of which the variable retains its value in memory
- a temporal concept

...but they can be related

Example: A Java variable declared with a method that contains no method calls

What is the scope and lifetime of the variable sum?

Scope: its declaration to the end of the method

Length: when the method is entered and up to when the method terminates

```
public int addNumbers (int x, int y) {
    int  sum = x + y;
    return sum;
}
```

...or unrelated

Example: A C and/or C++ variable declared in a function using the specifier **static***
**statically bound to the scope of that function and is also statically bound to storage*

What is the scope and lifetime of the variable counter?

Scope: within the function **countCalls ( )**

Length: from start to end of the program

```
void countCalls ( ) {
  static int counter = 0;
  counter++;
}
int main ( ) {
  countCalls ( ) ;
  countCalls ( ) ;
}
```

...or unrelated

Example: C++ subprogram calls

What is the scope and lifetime of the variable sum?

Scope: within the function **compute ( )**; does not extend to **printheader ( )**

Length: from start to end of the program

```
void printheader ( ) {
  . . .
} /* end of printheader */
void compute ( ) {
  int sum ;
  . . .
  printheader ( ) ;
} /* end of compute */
```

referencing environments

- The collection of all names that are visible in the statement
- *At this point in the program, what variables can be referenced?*
- Referencing environments differ between static scoped languages and dynamic-scoped languages

referencing environments: static

In a static-scoped language, it is the local variables plus all of the visible variables in all of the enclosing scopes

Point	*Referencing Environment*
1	local a and b (of sub1), global g for reference, but not for assignment
2	local c (of sub2), global g for both reference and for assignment
3	nonlocal c (of sub2), local g (of sub3)

```
g = 3 ;                 # A global
def sub1():
  a = 5 ;               # Creates a local
  b = 7 ;               # Creates another local
  . . .                                                  1
  def sub2():
    global g ;          # Global g is now assignable here
    c = 9 ;             # Creates a new local
    . . .                                                2
  def sub3():
    nonlocal c :        # Makes nonlocal c visible here
    g = 11 ;            # Creates a new local
    . . .                                                3
```

referencing environments: dynamic

In a dynamic-scoped language, it is the local variables plus all visible variables in all active subprograms

A subprogram is active if its execution has begun but has not yet terminated.

Point	*Referencing Environment*
1	a and b of sub1, c of sub2, d of main, (c of main and b of sub2 are hidden)
2	b and c of sub2, d of main, (c of main is hidden)
3	c and d of main

```
void sub1 ( ) {
   int a, b;
   . . .                    1
} /* end of sub1 */

void sub2 ( ) {
   int b, c;
   sub1( ) ;
   . . .                    2
} /* end of sub2 */
void main ( ) {
   int c, d;
   sub2 ( ) ;
   . . .                    3
} /* end of main */
```

named constants

- A variable that is bound to a value only once
- Useful for readability and program reliability (through parameterizing the program)

```
const int MAX_USERS = 100;

final int threshold = input * 2;
```

named constants

```
void example ( ) {
    int [ ] intList = new int[100];
    String [ ] strList = new String[100];
    for (index = 0; index < 100; index++) { ... }
    for (index = 0; index < 100; index++) { ... }
    . . .
    average = sum / 100;
    . . .
}
```

- Need to manually update every line = error prone

```
void example ( ) {
    final int len = 100;
    int [ ] intList = new int[len];
    String [ ] strList = new String[len];
    for (index = 0; index < len; index++) { ... }
    for (index = 0; index < len; index++) { ... }
    . . .
    average = sum / 100;
    . . .
}
```

- Needs only one update
- Easier to read

named constant examples

Programming Language	*Keyword(s)*
Ada	constant
C++	const
Java	final
C#	const readonly

initialization

- The binding of a variable to a value at the time it is bound to storage
- Variables are usually initialized when on the declaration that creates the variable
- Can be static or dynamic

initialization

Static

- binding and initialization occur before run time
- the initial values can be specified as a literal or an expression whose only nonliteral operands are named constants that have already been defined

Dynamic

- initialization is also dynamic and
- the initial values can be any expression

THANK YOU